

Excluding Classical Cipher Families for the 1939 D’Agapeyeff Challenge: Key-Free Counts and Searches with Demonstrated Power

Chase Hendrick

Independent Researcher

chase@hendrickresearch.com

ORCID 0009-0002-9754-6087

Abstract

The cryptogram that Alexander d’Agapeyeff printed as a challenge in *Codes and Ciphers* (1939) has no accepted reading. Its 392 digits form 196 cells of a 5×5 Polybius square. Many searches have reported no signal, but few showed that their search could find a planted text of the same length, so a null result closed little. We separate two kinds of evidence. First, counts that no key can change: under any one-to-one letter key, with or without any transposition, the fewest single-cell errors needed to turn a text’s letter counts into the cells’ counts is half the distance between the sorted count vectors (proved). Among 479,051 windows of 196 letters of English prose the closest needs 8 such errors. In four-square the number of symbols each side of a pair can use is fixed by the plaintext and the plain squares, whatever the cipher squares (proved); with standard plain squares English never reaches the cells’ 13 and 18, but with chosen plain squares it does, so keyed four-square and two-square stay open. Second, compiled annealing searches whose power is shown on planted held-out texts, with shuffled cells as the control: width-14 columnar transposition with a letter key (18 of 18 planted texts recovered), four-square with standard plain squares, a repeating coordinate shift on a keyed square at periods 2 to 14, homophonic keys, the book’s own dummy rule, 363 reorderings that earlier work flagged, and enciphering errors at the rate of the book’s worked example. In every family the cells score far below planted text, and in all but two comparisons, which the text discusses, within the range of their shuffles. A screen of 39 languages by letter counts finds Latin closest (4 errors at its closest window); searches under Italian, Latin, Catalan and Romanian models also find nothing, including Latin under a 14-column key. Finally, taking van Eykelen’s no-message recipe as the null hypothesis, a test of thirty order statistics that flags 19 of 20 planted messages under a keyed square finds the cells’ order indistinguishable from a no-message dealing; the test is blind to a turning grille, which remains open. All results except the stated propositions are numerical. No reading is claimed, and a construction with no message remains consistent with every measurement.

1 Introduction

The first edition of Alexander d’Agapeyeff’s *Codes and Ciphers* [6] ends with a cryptogram of 395 digits offered as a challenge to the reader. It was dropped from later editions, Barker discussed it in *Cryptologia* in 1978 [4], and it has no accepted reading. The digit block used here is the one printed on the English Wikipedia page for the cipher [23], fetched on 2 October 2026; we did not inspect a copy of the 1939 book. Without its final three digits, 000, the block has 392 digits. Read in pairs, every pair has a first digit from $\{6, 7, 8, 9, 0\}$ and a second from $\{1, 2, 3, 4, 5\}$, so the text is 196 cells of a 5×5 Polybius square, the system the book teaches with a worked example. Laid out as a

14 × 14 grid, five of the cells’ symbols occur only in the last column, an observation due to Pelling [15] and quantified by Snider [19].

Much has been tried. Gariazzo [7, 8] rebuilt a quadgram solver on 2.5 to 3.29 million characters and searched substitution after fixed routes, free column orders, a 14 × 14 turning grille and double transposition, with a shuffled-cell baseline; the baseline’s best score beat the cells’. Van Eykelen [21, 22] found a procedure with no plaintext, a tally of 18 pairs dealt onto the grid by hand, that reproduces fourteen statistics of the cells. Marland [13] ran several hundred configurations over seven methods. Rodrigues, in comments on [15], reported that a transposition-then-substitution solver needed about 240 letters for a 14-column key. Pelling [16] found that the book’s double-transposition example is copied from Kerckhoffs [9] with the key numbered in the decryption direction. Several readings have been published [20, 5, 12]; the companion repository checks eight of them against counts the cells force, and none passes as a full-length reading. Earlier commentary includes Shulman’s note in *The Cryptogram* of 1952 [18], Schmech’s write-up of the cipher as an open challenge [17], a blog series that tested an ADFGX reading [2], and one that considered the book itself as a dictionary-code key and noted that the cryptogram was still printed in a 1949 reprint and dropped only in 1952 [1]. A 2026 repository [3] argues from the cells’ index of coincidence, 0.0697 against about 0.0667 for English, that the cipher is a monoalphabetic Polybius substitution under a transposition. The index cannot settle that: it is unchanged by any one-to-one key, and Section 3 shows that the cells’ letter counts, which such a key would keep, are at least 8 errors from those of any of 479,051 windows of English prose.

What most of this work lacks is a demonstration of power. A search that cannot find a planted 196-letter text under the hypothesis it tests closes nothing when it finds nothing on the cells. Some families can instead be closed without searching keys, by a statistic no key can change. This note separates the two kinds of evidence and reports both. It adds four things. First, three propositions about counts that a key cannot change, or changes only in part (Section 3), with the measurements they license: letter counts under any one-to-one key and transposition, the fewest enciphering errors that would reconcile them, and the side counts of four-square, which depend on the plain squares. Second, compiled searches whose power is shown on planted held-out texts before they are run on the cells, with shuffled cells as the control (Section 4). Third, a test of whether enciphering errors at the rate of the book’s own worked example could hide English, and a screen of 39 languages by letter counts, followed by searches in Italian and in Latin, the closest language (Section 5). Fourth, an account of what remains open (Section 7).

Propositions 1–3 are proved. Every other result is numerical, computed with fixed seeds, and none is stated as a theorem. No reading of the cells is claimed, and no letter string produced by any search on the cells is recorded.

2 Data, model and conventions

The cells. Write each cell as a number $x \in \{0, \dots, 24\}$, $x = 5r + c$, with the row r given by the first digit in the order 6, 7, 8, 9, 0 and the column c by the second digit in the order 1, 2, 3, 4, 5. The 196 cells use 18 of the 25 symbols. Their counts, sorted, are 20, 17, 17, 17, 17, 16, 15, 14, 12, 12, 11, 11, 9, 3, 2, 1, 1 and 1: thirteen common symbols with nearly equal counts and five rare ones, which occur only in the last column of the 14 × 14 grid. Reordering the digits of every printed group of five changes the pairs. Five reorderings keep all 196 pairs on the square; only one of them, positions 0, 1, 4, 3, 2, also keeps the final filler 000 last. It is called the regrouping below and is searched alongside the printed cells where noted.

Language model. Candidate plaintexts are scored by a quadgram model with interpolated absolute discounting [14] (discount 0.75, backing off to trigram, bigram and smoothed unigram), fit on 1,916,398 letters of public-domain English prose with J folded into I to match the 25 cells. A score is the mean log probability per letter, in nats, over all quadgrams of the text. A model that sent unseen contexts to a flat 1/26 let nonsense keys win in development, which the back-off prevents.

Held-out text. Planted texts are windows of 196 letters from three texts that do not occur in the model’s training data: Austen (12,799 letters), Doyle (7,592) and Wells (16,000). Windows whose true text scores below -2.5 a letter, such as Project Gutenberg header text, are redrawn where stated; the number redrawn is recorded with each result.

Searches. Each search is simulated annealing [10] in a compiled C kernel called through Python, with a linearly falling temperature, several independent restarts, and a fixed seed per run. A planted text counts as recovered when at least 90 percent of its letters are right (95 percent for four-square, and every letter for the searches at widths 1 to 7); a search can find a slightly different key that scores higher than the true one, which is a property of the model, not of the search.

Controls. Shuffles keep the cells’ symbols and destroy their order. A shuffle of all 196 cells is given exactly the search and the transformations the cells receive. When a family contains many variants, the cells’ best over the family is compared with each shuffle’s best over the same family, so that the comparison accounts for the number of variants tried.

3 Counts no key can change

Proposition 1. *Let a plaintext p of length N over an alphabet A be enciphered by any transposition followed by any injective map $\kappa: A \rightarrow S$. Then the multiset of symbol counts of the ciphertext equals the multiset of letter counts of p .*

Proof. A transposition permutes positions and leaves the count of every letter unchanged. An injective κ sends the positions holding a letter a to positions holding $\kappa(a)$ and no others, so the count of $\kappa(a)$ in the ciphertext is the count of a in the transposed text. $\square$

Proposition 2. *Let $u, v \in \mathbb{Z}_{\geq 0}^n$ with $\sum_i u_i = \sum_i v_i$, and let $u^\downarrow, v^\downarrow$ be their entries sorted in nonincreasing order. Call an error the replacement of one symbol by another at one position. The fewest errors that turn a text with symbol counts u into a text whose symbol counts equal v up to a relabeling of the symbols is $D(u, v) = \frac{1}{2} \sum_i |u_i^\downarrow - v_i^\downarrow|$.*

Proof. An error lowers one count by one and raises another by one, so it changes $\sum_i |u_{\sigma(i)} - v_i|$ by at most 2 for every fixed relabeling σ . Hence at least $\frac{1}{2} \min_\sigma \sum_i |u_{\sigma(i)} - v_i|$ errors are needed. For nonnegative reals the minimum over permutations of $\sum_i |a_{\sigma(i)} - b_i|$ is attained when both sequences are sorted the same way: if $a_{\sigma(i)} < a_{\sigma(j)}$ while $b_i > b_j$, exchanging $\sigma(i)$ and $\sigma(j)$ does not increase the sum, by the four-case check of $|a - b| + |a' - b'| \leq |a - b'| + |a' - b|$ for $a \leq a'$, $b \leq b'$; finitely many exchanges reach the sorted matching. So at least $D(u, v)$ errors are needed. With the sorted matching fixed, the symbols whose count exceeds its target have a total surplus of exactly $D(u, v)$, equal to the total deficit, and moving one surplus position at a time to a symbol in deficit reaches the target in $D(u, v)$ errors. $\square$

Proposition 2 is a best case for the errors: they must all be chosen to move the counts toward the cells. Combined with Proposition 1, it gives the fewest errors with which a plaintext under any transposition and any one-to-one key could have produced the cells' counts.

Proposition 3. *In four-square, let the plain squares place letters at cells $(r_1(a), c_1(a))$ and $(r_2(a), c_2(a))$, and let the cipher squares U and L be bijections from the 25 cells to 25 symbols. A plaintext pair (a, b) is enciphered as $(U(r_1(a), c_2(b)), L(r_2(b), c_1(a)))$. For any plaintext, the number of distinct first symbols equals the number of distinct pairs $(r_1(a), c_2(b))$ over its pairs, and likewise for the second symbols; neither depends on U or L .*

Proof. U is a bijection, so two first symbols coincide exactly when their arguments coincide; the same holds for L . $\square$

The side counts depend on the plain squares, which are part of the key. Proposition 3 makes them key-free only when the plain squares are the standard alphabet. In horizontal two-square each cipher square is a plain square, so nothing drops out and the proposition gives no exclusion. When the cipher symbols are written as Polybius cells, as here, the cells' labels are one more bijection and change nothing.

Measurements. Over 479,051 windows of 196 letters of the English training prose, taken every fourth letter, none is as flat as the cells, uses as few letters and has a largest count as small, all three together; the closest window by Proposition 2 needs 8 chosen errors, and over 5,116 windows of the held-out texts the fewest is 15 and the median 29.

For four-square, the cells' two sides use 13 and 18 distinct symbols, at either phase of pairing. Held-out English under the standard plain squares never uses fewer than 21 on its larger side (370 windows), and with both plain squares drawn at random never fewer than 20 (7,400 draws, 0 reaching the cells). Chosen plain squares are another matter: climbing both plain squares toward the cells' side counts reaches them exactly in 15 of 15 held-out windows. The count therefore excludes four-square with standard plain squares, and with plain squares drawn independently of the text, but not four-square with chosen plain squares, nor two-square. The regrouping uses 21 and 19, which this count does not exclude.

A repeating shift of the row and column coordinates modulo 5 on a keyed square spreads each letter over several cells. Over 16,470 draws of held-out English windows under random squares and random shift keys of periods 2, 3, 4, 5, 7 and 14, shifting both coordinates, the column only or the row only, no draw uses fewer than 20 symbols, and none has both the cells' 18 symbols and 177 cells in its three busiest rows. This count assumes a key drawn at random; a key that repeats one shift is a plain square, covered by Proposition 1.

Three further counts exclude simple readings outright. If the five rare symbols were word spaces, the cells would hold 188 letters in 8 words (the last cell is itself a rare symbol), 23.5 letters a word, using 13 letters; over 669 runs of held-out English of that length the longest mean word is 8.5 letters and the fewest letters used is 19. If only the column digit carried the message, the 196 column digits paired would be a substitution of 98 letters; their rate of coincidence is 0.0423 and 0.0369 at the two phases, below every one of 373 English windows of that length (lowest 0.0503). A code giving each letter one fixed pair of cells has at most 25 different pairs, while the cells have 79 and 80.

4 Searches with demonstrated power

Table 1 lists each searched family with its planted recovery, the score of its weakest planted text, the cells' best score and how many shuffles reach it. The rows are summarised below.

Table 1: Searches with demonstrated power. “Planted found” counts planted texts recovered with at least 90 percent of letters right. “Weakest planted” is the true-key score per letter of the weakest planted text; in rows with slips or wrong cells it is the lowest score at which such a text was found. “Cells’ best” is the best score per letter over the family on the printed cells and the regrouping together. ^a Family-wise: the cells’ best over all variants against each shuffle’s best over the same variants. The other entries compare the cells’ best case with its own shuffles. The repeating-shift row compares the best of 24 cases with 3 shuffles of that case only.

Family searched	Planted found	Weakest planted	Cells’ best	Shuffles as high
Keyed square; columnar widths 1, 2, 4, 7	8/8	−2.01	−3.82	1–4/4
Columnar width 14, both directions	18/18	−2.07	−3.25	7/20
Four-square	4/6	−2.02	−3.34	9/20
Repeating shift, periods 2–14	11/12	−2.08	−3.33	0/3
Nulls by place; reversed; column dig-its	6/6	−2.14	−3.28	0/8 ^a
Homophonic key, capped	6/6	−1.98	−2.86	3/8
Delays, nulls 2–14, rails, columns 10–28	4/4	−2.03	−3.28	7/8 ^a
Keyed square with 8 enciphering slips	3/3	−2.49	−3.90	–
Italian keyed square (0 and 8 slips)	8/8	−2.60	−4.56	5/8
Latin keyed square and dummy rule (0 and 8 slips)	12/12	−3.27	−3.94	7/8 ^a
Latin, columnar width 14 (0 and 8 wrong cells)	8/8	−3.26	−3.71	8/10
Latin, repeating shift, periods 2–14	15/18	−3.35	−3.99	1/3
Catalan keyed square and dummy rule (0 and 8 slips)	6/6	−2.33	−3.47	7/8 ^a
Romanian keyed square and dummy rule (0 and 8 slips)	6/6	−2.71	−3.19	13/40 ^a

Plain square and complete columnar transposition. Annealing the column order and the letter key together recovers 8 of 8 planted texts at widths 1, 2, 4 and 7, where the cells score −3.9680 to −3.8212 a letter. At width 14, which fits the 14×14 grid, a compiled kernel with 8 restarts of 16,000,000 steps recovers 18 of 18 planted texts in both directions, 6 of them with 8 wrong cells; the cells and the regrouping score −3.6429 to −3.2526, with 5 to 12 of 20 shuffles as high. An earlier pure-Python search had recovered none of three planted texts at this width.

Every order at small widths, any language. For columnar transposition in either direction with a short last row, and for periodic transposition, every order at widths 2 to 9 was scored by the mutual information of successive symbols, which no letter key can change, so the score is unchanged by any letter key; power was shown for English and German. In each of the 24 width and family cases, both planted texts beat the best order of a shuffle of their own symbols by more than the cells and the regrouping beat the best of three shuffles of theirs. Over all cases the planted margins are at least 0.0345 nats of information and the cells’ at most 0.0698, so the separation holds case by case, not across cases. The planted texts are compared with one shuffle and the cells with three, and at width 9 the planted margins are small, so power there is weak. At width 10 the method keeps power only for the direction called done, where planted texts beat their shuffles by at least 0.1319 and the cells by −0.0100. Double transposition with every pair of orders at widths 2 to 6, both passes in the same direction, leaves the cells below planted English and German in 50 of 50 cases. 379,521 keywords applied as single, double and square transpositions, 2,208,702 orders, bring

4 of 6 planted keywords to the top; the cells' best, 0.8161, is under a shuffled copy's 0.8290.

Four-square. With ten restarts of 1,000,000 steps the search recovers 4 of 6 planted texts. The cells score at best -3.5543 and the regrouping -3.3427 , with at least 9 of 20 shuffles as high in every case.

A repeating shift on a keyed square. The square and the shifts are annealed together, 4 restarts of 4,000,000 steps, at periods 2, 3, 4, 5, 7 and 14 with both coordinates or the column only shifted. Planted texts are recovered 11 of 12 times; the miss, a column shift of period 7, ended with 84 percent of letters right. The best of 24 cases on the cells and the regrouping is -3.3261 , the regrouping at period 14, where the key has the most freedom and only 3 shuffles were run; that comparison is weak, and the gap of more than one nat a letter to planted English is the result.

The book's dummies, and other short attacks. The book allows a dummy at every third, fourth or fifth letter [23]. Dropping every phase of those periods and of period 14 (one grid column), and reading the cells backwards, the keyed-square search recovers 6 of 6 planted texts. The cells' best over 28 variants is -3.2848 , a little above all 8 shuffles' best variants (-3.4798 to -3.3085). That edge does not survive the wider family below.

A homophonic key, and a trap. Under the default model a run of one repeated letter scores above English: in a development run recorded in the companion note, the cells with every symbol sent to O scored about -1.3 a letter. An unconstrained many-to-one key therefore collapses onto one letter, for the cells and for their shuffles alike. Refusing any key that gives one letter more than 36 places, the most the commonest letter takes in any 196-letter window of the held-out texts, the search recovers 6 of 6 planted homophonic texts. The cells score -3.1312 , with 6 of 8 shuffles as high, and the regrouping -2.8590 , with 3. We record the trap because a homophonic solver without such a limit can return fluent-looking runs of frequent letters on any input.

Old leads. Earlier probes, scored with an 8,689-letter model and without planted texts, left faint leads, notably a delay of 79 places between the row and the column digits that only 3 of 200 shuffles matched. 363 transforms were solved as keyed squares: every delay from 1 to 195, every phase of nulls at periods 2 to 14, rail fences of 2 to 14 rails and rows of 10 to 28 read down, each undone and done. Planted texts are recovered 4 of 4. The cells' best over all transforms is -3.2848 , and 7 of 8 shuffles reach a best at least as high; within each of the four families 7 or 8 of 8 do. The largest single excess over a transform's own shuffles is delay 115, $z = 3.41$ among 363 comparisons with 8 shuffles each, what the largest of so many noisy comparisons looks like.

5 Enciphering errors and other languages

Errors. The book's worked example has about four faults in 92 letters: 89 pairs for 92 letters, and AREA written as ARYA [23]. At that rate the challenge might carry about eight. Most of those faults are omissions, which Proposition 2 does not count directly, but a 196-letter window of the intended passage differs from the surviving text by at most $d + k$ substitutions when d letters were dropped or added and k changed, so the substitution minimum over windows is still a lower bound. Under a one-to-one key, Proposition 2 answers the best case: no window of the held-out texts reaches the cells' counts with fewer than 15 chosen errors, and the closest window of the training prose needs 8. Random errors do worse. A digit slip, a wrong row or column digit, moves a

Table 2: The twelve treebanks closest to the cells’ letter counts. “Fewest” and “Median” are the fewest errors (Proposition 2) at the closest and at the median window of 196 letters; “Within 8” counts windows needing 8 or fewer.

Treebank	Letters	Fewest	Median	Within 8
Latin (ITTB)	2,204,065	4	22	294
Catalan (AnCora)	2,171,416	5	28	8
Latin (Perseus)	140,605	6	23	82
Romanian (RRT)	983,284	7	28	2
Welsh (CCG)	208,261	8	26	4
French (GSD)	1,619,253	8	28	2
Estonian (EWT)	408,673	8	29	2
Norwegian (Bokmaal)	1,342,058	8	29	2
Faroese (FarPaHC)	142,591	9	25	0
Icelandic (Modern)	355,360	9	27	0
Spanish (GSD)	1,784,269	9	27	0
Swedish (Talbanken)	465,192	9	28	0

letter to one of the eight cells sharing its row or column; with 4 to 48 slips or free errors, 286,496 draws in all, no draw reaches the cells, and at least 13 further errors are always needed. The search is not blinded by such errors either: planted English with 8 slips is found at -2.4865 a letter or better with at least 95 percent of letters right, against the cells’ -3.9003 under the same search, and planted English falls to the cells’ level only at 48 errors, where 24 percent of its letters are right.

A screen of languages. Proposition 2 needs no language model, so it can screen languages. Table 2 gives the fewest errors from windows of real text in 39 languages to the cells’ counts. The texts are the sentences of Universal Dependencies treebanks; accents are removed, a few letters are spelled out, Russian is transliterated in a plain British style, and J is folded into I. Latin stands out: its closest window needs 4 errors, its median is 22 against at least 25 for every other language, and 294 windows need 8 or fewer; classical Latin needs 6 at its closest. French needs 8, German 12, and transliterated Russian, the author’s first language, 13. A count fit is a reason to search, not a reading.

Italian and Latin. An earlier pass that let each of seven languages choose a merged pair of letters had preferred Italian. With Manzoni’s *I promessi sposi* of 1827, 1,026,120 letters, the closest of 256,482 windows needs 9 errors and the median 28. Under a quadgram model fit on its first 30 chapters, planted Italian from the remaining chapters is recovered 4 of 4 times, and 4 of 4 with 8 slips; the cells score -4.5563 , with 5 of 8 shuffles as high. For Latin a model was fit on 1,983,658 letters of the Index Thomisticus. Planted Latin from its held-out tail and from classical Latin, which the model never saw and scores as low as -3.3036 , is recovered 6 of 6 times, and 6 of 6 with 8 slips. The cells as a keyed square score -4.5513 (5 of 8 shuffles as high), and with the book’s dummy rule at best -3.9424 , reached by 7 of 8 shuffles’ best variants.

Latin under other systems. Because a transposition keeps letter counts, the width-14 joint search of Section 4 was rerun under the Latin model, starting from a Latin frequency key. It recovers 8 of 8 planted Latin texts in both directions, with 0 or 8 wrong cells and including classical Latin; the printed cells score -3.7074 with the columns undone (8 of 10 shuffles as high) and -4.0104 done (6 of 10). The repeating-shift count of Section 3 does not exclude Latin: Latin uses so few letters that its draws reach as few as 16 symbols, and 1 of 9,774 draws reaches the cells. The joint search

Table 3: Power of the no-message test. Planted texts flagged at a family-wise p-value of 0.05 or less, out of 20 per system. A random transposition of the planted text is itself a no-message dealing, the calibration case.

Message system	Flagged
Keyed square (any one-to-one key)	19
14-column transposition, done	19
Repeating coordinate shift, periods 2 to 14	10
Turning grille, 14×14	3
14-column transposition, undone	2
Random transposition (calibration)	1

was therefore run under the Latin model at periods 2 to 14, shifting rows, columns or both: it recovers 15 of 18 planted texts (the misses are search failures), the cells’ best of 36 cases is -3.9855 , and in 5 cases the cells beat all three of their shuffles, where chance gives about nine. Four-square with standard plain squares under the Latin model recovers 3 of 3 planted texts, with 7 of 8 shuffles reaching the cells; a capped homophonic key recovers only 2 of 3, so that result is weaker.

Catalan and Romanian. The next two languages of Table 2 were searched as Latin was, under the keyed square and the book’s dummy rule, with models fit on their treebanks. Planted text is recovered 3 of 3 in each, also with 8 slips. For Catalan the cells’ best, -3.4667 , is reached by 7 of 8 shuffles. For Romanian it was reached by only 1 of 8 at first; with 40 shuffles, 13 reach the cells’ best and 12 the regrouping’s, so the first result was the small sample.

6 Starting from no message

Van Eykelen’s recipe [22] can be stated as a null hypothesis and tested. Under it the cells are their own symbols dealt onto the grid at random, the rare symbols at row ends; here the 8 rare cells are held in place and the other cells are dealt at random. The letter counts are then fixed, and only the order can separate a message from no message. Thirty order statistics, declared before any score was read and unchanged by any one-to-one key, were used: the share of equal symbols and the mutual information of symbols at each lag from 1 to 14, the number of different neighbouring pairs and the number of repeated triples. Each statistic gets a two-sided rank p-value against 20,000 dealings, and the family-wise p-value is the share of dealings whose smallest p-value is as small as the cells’.

The cells give a family-wise p-value of 0.2814. Their smallest single p-value, 0.0138, is the mutual information at lag 3, the trace van Eykelen reported; it does not survive the correction for thirty statistics, and finding it again on the same cells is not independent confirmation. Table 3 gives the test’s power: planted held-out English hidden by five message systems, each tested against dealings of its own symbols.

The order of the cells is what a no-message dealing gives, by a test that would almost always have caught a message under a keyed square or a done 14-column transposition. The test cannot see systems that scramble order thoroughly: the turning grille and the undone 14-column transposition. The second is closed by the joint searches for English and Latin. Among the systems examined here, the order therefore leaves two explanations: no message, or a message under a system that destroys order as completely as a turning grille. The first is simpler; neither is proved.

7 What remains open

The width-14 transposition was searched under English and Latin models; other languages under it are open, and the small-width searches of Section 4 hold for any language. The 14×14 turning grille, the main message system the order of Section 6 still allows, is not closed: a compiled search recovers 4 of 4 planted grilles with the letter key given, but 0 of 4 with grille and key both unknown, at most 18 of 49 holes right after 8,000,000 steps, so the cells were not searched with it. Four-square with keyed plain squares and two-square are not excluded by the count, and a search annealing all four squares (with the move set of the two-square solver in Colossus) recovered 0 of 6 planted texts after 40,000,000 steps, its wrong keys reaching -2.5078 a letter; at 196 letters it has no power, so the cells were not searched. A general table from letter pairs to cell pairs, such as a 2×2 Hill cipher over a keyed square, is not excluded by the pair count, which is ordinary for prose (29,336 and 11,539 windows have as many different pairs as the cells). Transposition beyond width 10, other than at width 14, is open, as is double transposition above width 6, for which divide-and-conquer methods exist [11]. Finally, a construction with no message, such as van Eykelen’s tally-and-deal recipe [22], is consistent with every count and score reported here, and passes the order test of Section 6.

8 Limitations

The digits are taken from a secondary transcription [23]; we did not inspect the 1939 book, whose page numbering differs between sources. The counts in Section 3 are taken over windows of particular corpora, and English prose of other kinds, such as telegraphese with abbreviations, may vary more. The counts for the repeating shift and for four-square with keyed plain squares assume a key drawn independently of the text; chosen plain squares defeat the four-square count. The searches measure power on held-out prose of the same length; a plaintext very unlike that prose could be missed. The decision that a family is closed rests on two comparisons, planted text against the cells and the cells against their shuffles; the thresholds (90 percent of letters, a gap of about one nat a letter) are judgments and were not formally registered in advance, and several controls use few shuffles. The language screen uses mixed-genre treebank sentences, with one folding convention. Our survey of prior work is bounded by the sources listed, several of which were read through search summaries; absence from it does not show that a result is new.

9 Reproducibility

Every probe is a module of the companion repository, <https://github.com/ChaseHendrick/Undeciphered-Texts>, under `engine/`, with a test under `tests/` and a dated note under `docs/logs/`. Each writes its result once to `engine/data/swarm_cache/`, and later calls `read` that file; deleting a file reruns its search, which takes from seconds to about thirty minutes on four cores and needs a C compiler. Probes that use outside text (Manzoni, the treebanks) fetch it into an ignored folder and store only counts and scores. The numbers and tables of this manuscript are written by `papers/dagapeyeff-exclusions/code/make_numbers.py` from those files, whose SHA-256 hashes head `numbers.tex`; it refuses any file that claims a reading. The README of the manuscript’s folder gives the commands.

References

- [1] D’Agapeyeff cipher (part 1) and (part 2). NumberWorld blog,

- <https://numberworld.blogspot.com/2013/07/dagapeyeff-cipher-part-1.html> (2013) and <http://numberworld.blogspot.com/2015/07/d-cipher-part-2.html> (2015). Read through search summaries.
- [2] Is the D’Agapeyeff cipher the ADFGX cipher? Please Decipher Me blog, <http://pleasedecipherme.blogspot.com/2010/11/is-dagapeyeff-cipher-adfgx-cipher.html>, 2010. Read through search summaries; a sequel, “D’Agapeyeff Cipher: Searching For A Solution”, followed in January 2011.
 - [3] Ashley and Antigravity. Structural decoupling and information-theoretic resolution of the 1939 D’Agapeyeff cipher. <https://github.com/ajejfiejof/dagapeyeff-cipher-solver>, commit 2009392, 2026. README and PAPER.md read on 6 October 2026.
 - [4] Wayne G. Barker. The unsolved D’Agapeyeff cipher. *Cryptologia*, 2(2):144–147, 1978.
 - [5] Brandon Caillahua Mendoza. Zenodo record 10.5281/zenodo.20652502, 2026. Title not retrieved at writing; the quoted reading is recorded in the companion prior-work note.
 - [6] Alexander d’Agapeyeff. *Codes and Ciphers*. Oxford University Press, London, 1939. First edition. The challenge cryptogram was dropped from later editions.
 - [7] Ruben Gariazzo. The D’Agapeyeff cryptogram (1939): Anatomy, reconstruction of a solver, and statistical assessment. <https://doi.org/10.5281/zenodo.22057249>, 2026.
 - [8] Ruben Gariazzo. Data and code of computational cryptanalysis of the D’Agapeyeff cryptogram (1939). <https://doi.org/10.5281/zenodo.21970478>, 2026.
 - [9] Auguste Kerckhoffs. *La Cryptographie Militaire*. Librairie Militaire de L. Baudoin, Paris, 1883.
 - [10] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
 - [11] George Lasry, Nils Kopal, and Arno Wacker. Solving the double transposition challenge with a divide-and-conquer approach. *Cryptologia*, 38(3):197–214, 2014.
 - [12] Keenan Leggett. Zenodo record 10.5281/zenodo.17993736, 2025. Title not retrieved at writing; the quoted reading is recorded in the companion prior-work note.
 - [13] Tim Marland. dagapeyeffresearch.com. <https://dagapeyeffresearch.com/>, 2026.
 - [14] Hermann Ney, Ute Essen, and Reinhard Kneser. On structuring probabilistic dependences in stochastic language modelling. *Computer Speech and Language*, 8(1):1–38, 1994.
 - [15] Nick Pelling. The d’Agapeyeff cipher once again... Cipher Mysteries, 23 December 2013, <https://ciphermysteries.com/2013/12/23/dagapeyeff-cipher>, 2013. Including reader comments by T. Rodrigues and J. Melichar, May and June 2014.
 - [16] Nick Pelling. A new clue for the d’Agapeyeff challenge cipher...? Cipher Mysteries, 5 March 2017, <https://ciphermysteries.com/2017/03/05/new-clue-dagapeyeff-challenge-cipher>, 2017.

- [17] Klaus Schmeh. The D’Agapeyeff cipher: An unsolved cryptogram from 1939. MysteryTwister challenge, <https://mysterytwister.org/media/challenges/pdf/mtc3-schmeh-02-agapeyeff-en.pdf>. Title from search results.
- [18] Ab Struse (David Shulman). The D’Agapeyeff cryptogram: A challenge. *The Cryptogram*, 1952. April/May 1952. Title from search results; not opened.
- [19] Billy Snider. dagapeyeff-col14. <https://github.com/billysnider-collab/dagapeyeff-col14>, 2026.
- [20] Ender Uygun. Zenodo record 10.5281/zenodo.18639390, 2026. Title not retrieved at writing; the quoted reading is recorded in the companion prior-work note.
- [21] Erik van Eykelen. Unmasking the D’Agapeyeff cipher (XIV, pessimistic take). MsgTrail, <https://msgtrail.com/posts/unmasking-the-dagapeyeff-cipher-xiv-pessimistic-take>, 2026.
- [22] Erik van Eykelen. Unmasking the D’Agapeyeff cipher (XV, the recipe, not the message). MsgTrail, <https://msgtrail.com/posts/unmasking-the-dagapeyeff-cipher-xv-the-recipe-not-the-message>, 2026.
- [23] Wikipedia contributors. D’Agapeyeff cipher. https://en.wikipedia.org/wiki/D%27Agapeyeff_cipher. Fetched 2 October 2026; digit block and worked exercise used as transcribed there.